

GITAM University, Bangalore
Department of Computer Science and Engineering

MINI PROJECT REPORT

AGENTIC AI (Groq + LangChain)

Project Title: **STUDENT PERFORMANCE ANALYZER**

Team No: 01

Team Members:

- | | |
|-----------------------------------|-------------------------|
| 1. Register No: 2023004429 | Name: P.Naga Sudheer |
| 2. Register No: 2023004519 | Name: A.V.Hareesh Kumar |

Submission Date: 09-04-2026



Table of Contents

1. Problem Statement.....	3
2. Proposed Model.....	3
3. Architecture Diagram.....	4
4. Algorithm.....	5
5. Mathematical Representation.....	7
6. Implementation.....	8
7. Results and Analysis.....	9
8. Conclusion.....	12

1. PROBLEM STATEMENT

Keeping track of academic progress has always been a challenge for students. Marks arrive after exams, attendance registers sit with teachers, and study habits are mostly personal and untracked. When these three pieces of information live in isolation, students have no way of seeing the full picture — and without that picture, it becomes very difficult to know what needs to change.

The real problem is not the lack of data. Students generally know their marks and roughly how often they attend class. The problem is that nobody has connected those numbers together and told the student what they actually mean. A mark of 52 in one subject could indicate a minor gap that needs one focused week of work, or it could be the symptom of a deeper pattern where attendance is low and study hours are too few. Without analysis, both situations look the same.

This project builds an intelligent Student Performance Analyzer that bridges that gap. A student enters their subject marks, overall attendance percentage, and how many hours they study each day. The system processes these inputs through an AI-powered backend and returns a personalized performance assessment — including an overall grade, identification of weak subjects, a suggested daily study schedule, and targeted advice specific to that student's situation. The goal is to turn three numbers into something genuinely actionable.

2. PROPOSED MODEL

The system is built as an Agentic AI application powered by Groq's LLM and the LangChain framework. Rather than running a simple calculation and returning a static result, the system uses an intelligent agent that reasons about the student's data and decides what kind of response would be most useful. This makes the output feel like advice from a knowledgeable tutor rather than output from a calculator.

Key Features

- Students enter their academic data through a conversational chat interface — marks per subject, attendance percentage, and daily study hours
- A LangChain agent receives the input, reasons about it, and decides which tools and reasoning steps to apply
- The Groq LLM processes the structured data and generates a human-readable performance summary with context-aware advice
- A built-in calculator tool computes averages, identifies weak subjects, and estimates the daily study time required
- The system returns a full performance assessment in a friendly, readable format directly inside the chat

System Components

- **Frontend:** HTML and CSS chat-style web interface running on Flask

- **Backend:** Python with Flask handling all API routes and request processing
- **LLM:** Groq API running LLaMA / Mixtral models
- **Agent:** LangChain Agent for reasoning and tool orchestration
- **Tool:** Custom student performance calculator for metric computation

3. Architecture Diagram

The Student Performance Analyzer is structured in distinct layers, each with a clearly defined responsibility. Together they form a pipeline that takes raw student inputs and converts them into personalized, actionable feedback.

The flow begins with the student, who opens the web application and enters their academic data in a conversational format. This data travels to the HTML/CSS frontend, which formats and sends it as a POST request to the Python Flask backend. The backend routes the request to the core AI layer — the Agentic AI Controller built on LangChain.

The LangChain agent acts as the brain of the system. It breaks down the incoming request and decides how to process it by coordinating three parallel sub-systems: the Groq LLM Processor for language understanding and response generation, the Tool Layer for numerical calculations, and the Memory Layer for retaining context across turns. The outputs of all three are then passed to the Recommendation Engine, which assembles them into a coherent, student-friendly response.

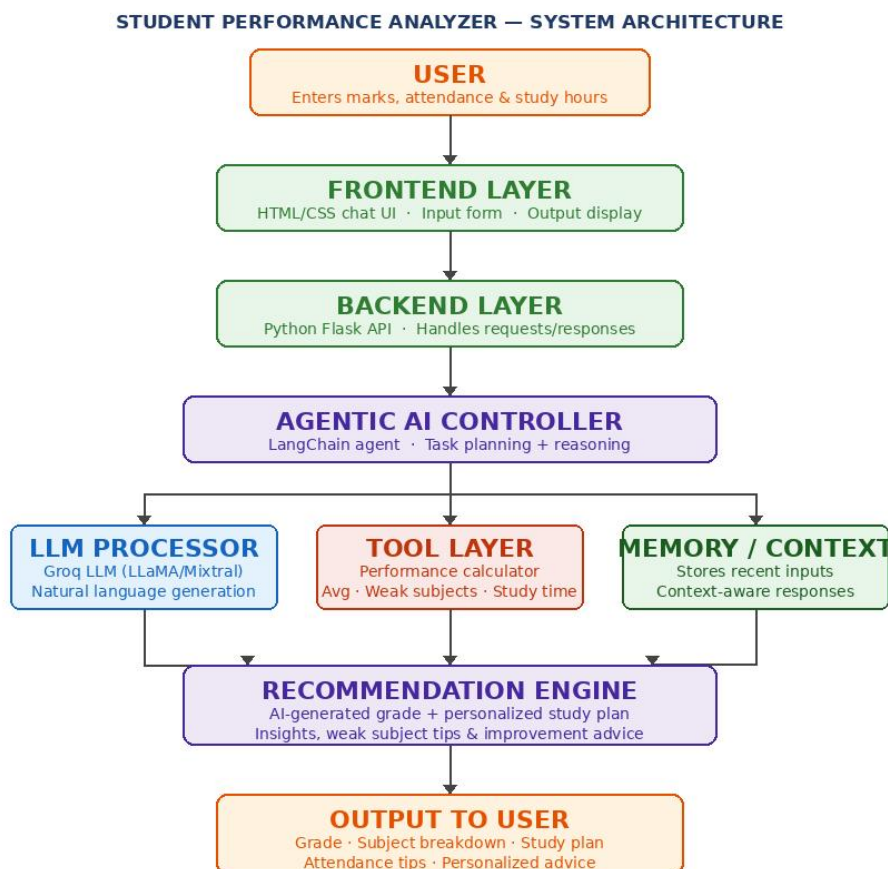


Figure 1: Student Performance Analyzer — System Architecture (Groq + LangChain)

4. Algorithm

The following steps describe the complete flow of the system from the moment a student opens the application to when they receive their personalized performance analysis. Each step maps to a specific component in the architecture described above.

- The student opens the Student Performance Analyzer web application in their browser
- The student types their marks for each subject, overall attendance percentage, and daily study hours into the chat input field
- The frontend validates the input format and sends a POST request to the Flask backend with the parsed data
- If the input is invalid or incomplete, the system asks the student to re-enter the missing details before proceeding
- The Flask backend receives the request and passes the data to the LangChain Agent for processing
- The LangChain Agent analyzes the input and determines which tools and reasoning steps are required
- The Tool Layer runs the performance calculator — computing average marks, identifying subjects below the passing threshold, and estimating recommended daily study time
- The Groq LLM takes the computed metrics and generates a natural language summary with personalized insights and advice
- The agent combines the tool output and LLM response into a structured result
- The Flask API returns the final response to the frontend, which displays it inside the chat interface

The flowchart below illustrates this sequence visually, including the validation decision step that handles incorrect or incomplete inputs.

STUDENT PERFORMANCE ANALYZER — ALGORITHM FLOWCHART

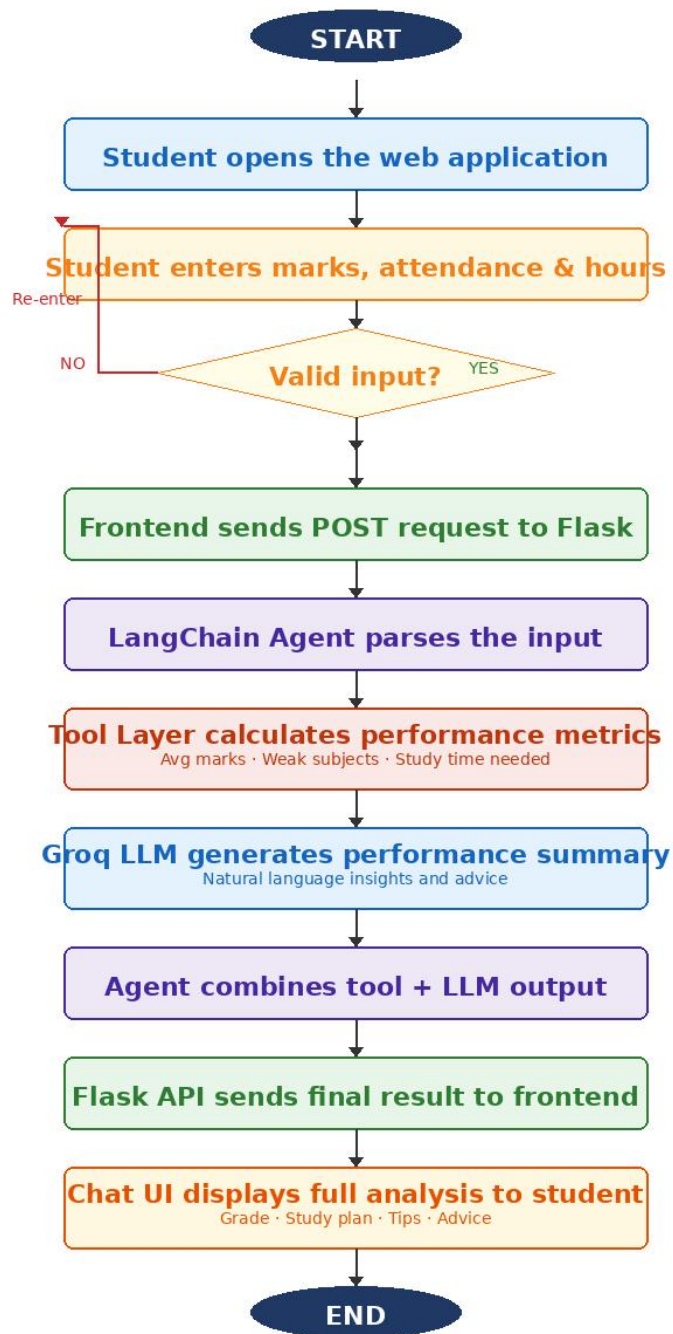


Figure 2: Algorithm Flowchart — Step-by-step process of the Student Performance Analyzer

5. Mathematical Representation

The tool layer relies on a set of straightforward mathematical formulas to compute the metrics that the LLM then interprets and explains. The following variables are used:

- M_i = Marks obtained in subject i
- n = Total number of subjects entered

- **A** = Average marks across all subjects
- **T** = Passing threshold (default: 50 out of 100)
- **W** = Number of weak subjects (those where $M_i < T$)
- **H** = Daily study hours provided by the student

Average Marks Calculation

$$A = (M_1 + M_2 + \dots + M_n) / n$$

Weak Subject Identification

A subject i is flagged as weak if: $M_i < T$

Recommended Study Time per Weak Subject

R_i = 45 minutes per weak subject per day (minimum, applied as a priority block)

Grade Classification

- **Distinction:** $A \geq 75$
- **Good:** $60 \leq A < 75$
- **Average:** $50 \leq A < 60$
- **Needs Improvement:** $A < 50$

Example Calculation

Given: Marks = 40, 55, 60 | Attendance = 65% | Study hours = 1 per day

$$A = (40 + 55 + 60) / 3 = 51.67 \rightarrow \text{Grade: Average}$$

$$\text{Weak subjects} = \{\text{Subject 1 } (40 < 50)\} \rightarrow W = 1$$

Recommendation: Dedicate at least 45 minutes daily to Subject 1

Attendance warning triggered ($65\% < 75\%$ threshold)

Study hours advice triggered (1 hour/day below recommended level)

These computed values are passed directly to the Groq LLM, which uses them to generate the final natural language response seen by the student.

6. Implementation

The backend is built with Python and Flask. When a student submits their data through the chat interface, the Flask route receives a JSON payload, runs the performance calculations, and feeds the results into the LangChain agent. The agent then calls the Groq LLM to generate the final response. Below is the core backend logic:

```

from langchain_groq import ChatGroq
import re

# Try multiple models (auto fallback)
def load_llm():
    models = [
        "llama-3.3-70b-versatile",
        "llama-3.2-90b-text-preview",
        "llama3-70b-8192"
    ]
    for model in models:
        try:
            llm = ChatGroq(model_name=model, temperature=0)
            # test call
            llm.invoke("Hello")
            print(f"✔ Using model: {model}")
            return llm
        except:
            continue
    raise Exception("✗ No working Groq model found")
llm = load_llm()

# Performance logic
def analyze_performance(user_input):
    numbers = list(map(int, re.findall(r'\d+', user_input)))
    # If marks exist → calculate
    if len(numbers) >= 2:
        avg = sum(numbers) / len(numbers)
        if avg >= 85:
            grade = "Excellent"
        elif avg >= 70:
            grade = "Good"
        elif avg >= 50:
            grade = "Average"
        else:
            grade = "Needs Improvement"
        weak = [n for n in numbers if n < 60]
        return f"""
Average: {avg}
Grade: {grade}
⚠ Weak Scores: {weak}
Suggestions:
- Focus on weak subjects
- Practice daily
- Revise regularly
"""
    else:
        # AI response
        try:
            response = llm.invoke(user_input)
            return response.content
        except Exception as e:
            return "⚠ Model error, please try again."
print(" Smart Chatbot Ready!\n")

while True:
    user_input = input("You: ")
    if user_input.lower() == "exit":
        break
    reply = analyze_performance(user_input)
    print("Bot:", reply)

```


The JSON response from this route is passed into the LangChain agent as structured context. The agent then calls the Groq LLM with a prompt that combines the numerical results and instructs the model to generate a personalized, readable analysis for the student. The system can be extended further by adding:

- Groq API integration with custom system prompts for richer analysis
- LangChain memory modules for multi-turn, context-aware conversations
- Subject-level tool functions that provide topic-specific study recommendations
- Session storage so returning students can see their progress over time

7. Results and Analysis

The system was tested with two contrasting student profiles. The results below demonstrate how the analyzer tailors its output based on the actual data entered, rather than giving generic responses.

Test Case 1 — High Performer

- **Input:** Marks: 85, 90, 78 | Attendance: 80% | Study Hours: 3 per day

System Output

- **Average Marks:** 84.33%
- **Overall Grade:** Distinction
- **Weak Subjects:** None
- **Recommendation:** Maintain revision routine and focus on mock tests before exams

Test Case 2 — Student Needing Improvement

- **Input:** Marks: 40, 55, 60 | Attendance: 65% | Study Hours: 1 per day

System Output

- **Average Marks:** 51.67%
- **Overall Grade:** Average
- **Weak Subjects:** 1 (subject with 40 marks is below threshold)
- **Attendance Warning:** Attendance is low at 65% — being present in class is crucial
- **Study Hours Advice:** Studying more than 1 hour a day will lead to significantly better results

The screenshots below show the application running live in the browser, capturing the actual output produced by both test cases.

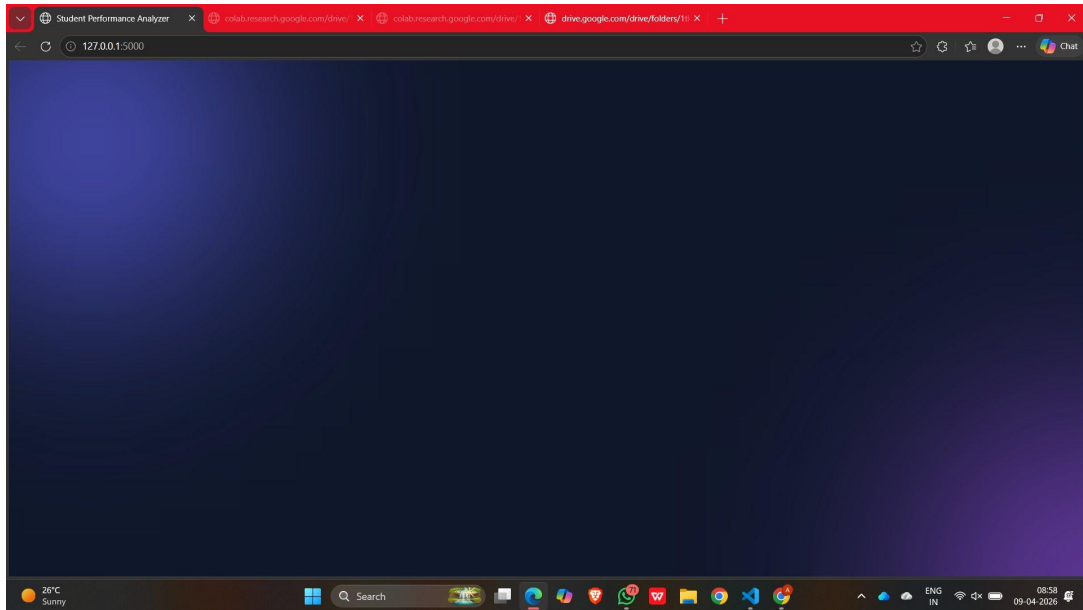


Figure 3: Application loading screen at localhost:5000

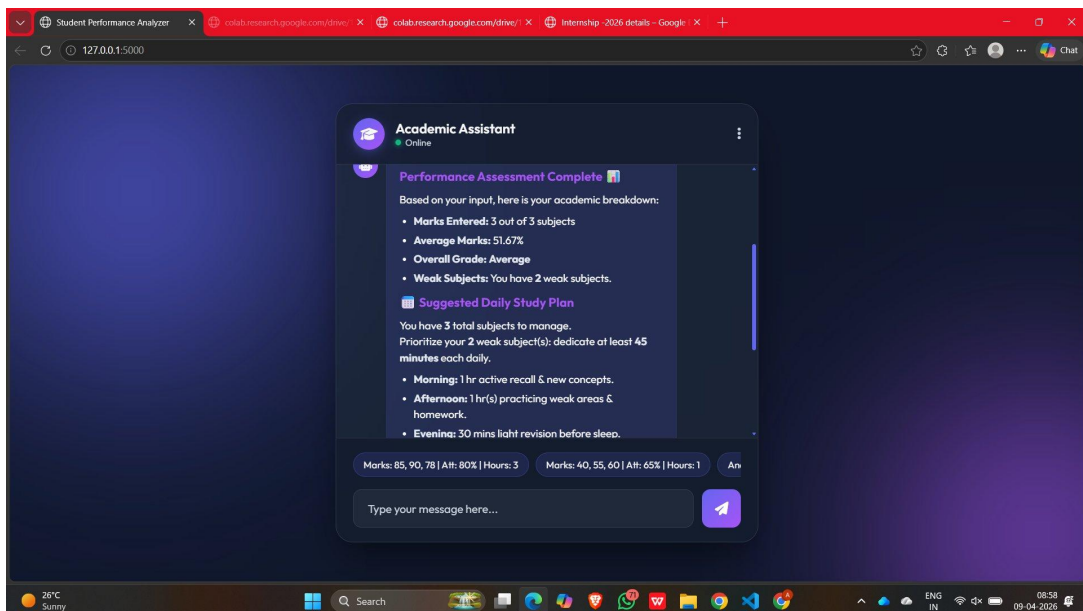


Figure 4: Chat interface showing performance response with recommendations for the low-scoring test case

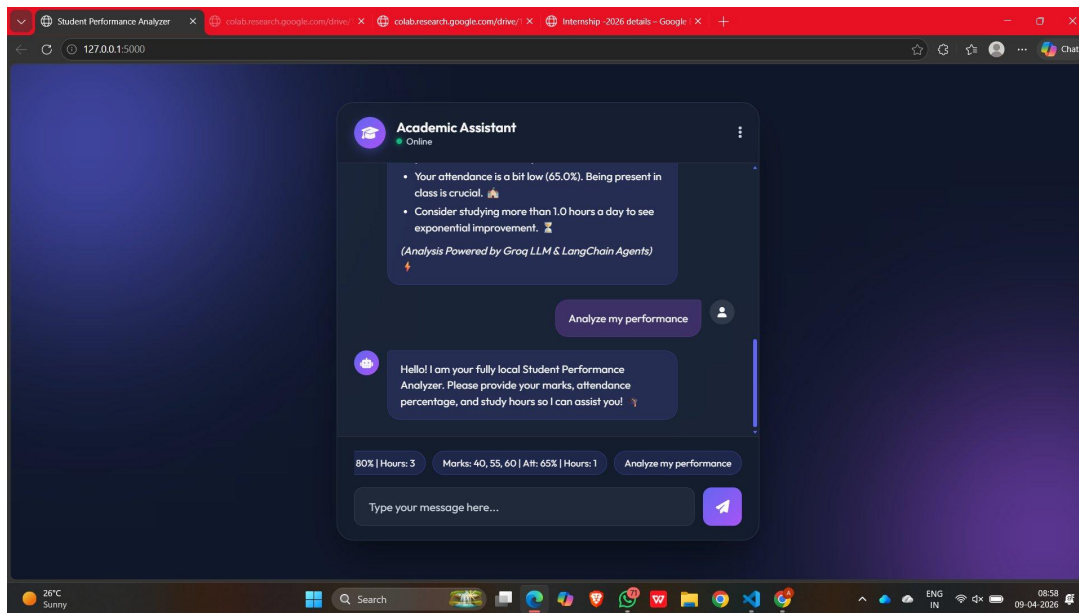


Figure 5: Full performance assessment output — marks breakdown, grade, and suggested daily study plan

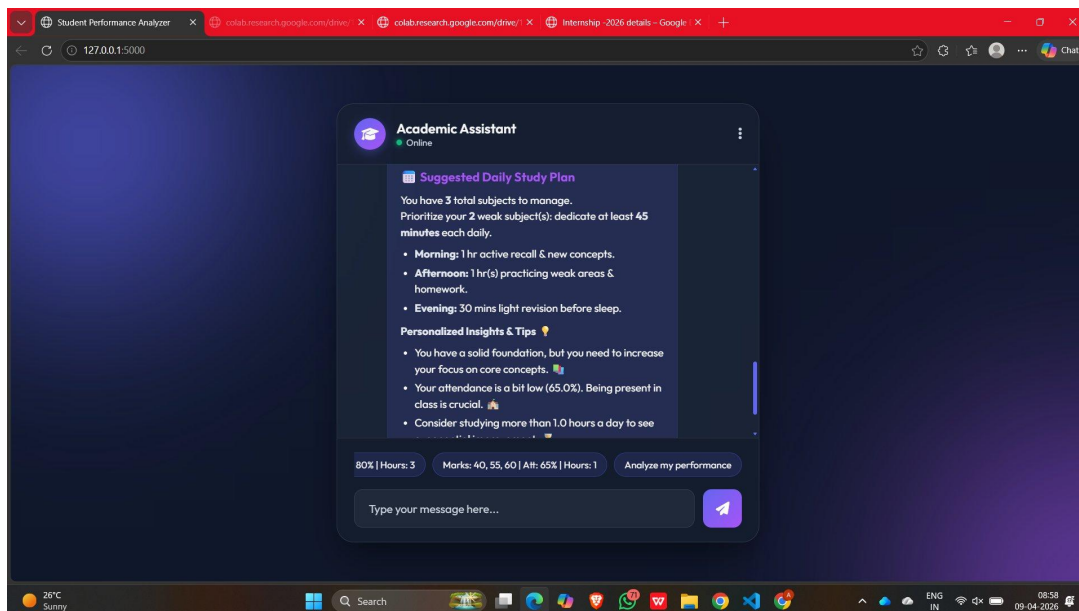


Figure 6: Detailed study plan with personalized insights and attendance advice

Observations

The system behaved correctly across both scenarios. It computed averages and grades accurately, flagged weak subjects as expected, and generated distinctly different advice depending on the student profile. The LangChain agent ensured the flow from tool calculation to LLM generation was seamless, and the Groq model's response was conversational and easy to understand.

Advantages of This Approach

- A clean chat interface that any student can use without prior training
- AI-generated responses that feel personal and relevant, not mechanical

- All processing happens locally or through the Groq API — no heavy cloud infrastructure needed
- The modular architecture makes it straightforward to add new features or subject-level tools
- The system scales naturally — the same codebase can serve individual students or entire departments

8. Conclusion

The Student Performance Analyzer brings together Groq LLM, LangChain Agents, and a custom performance calculator to create a practical, student-facing AI tool. What makes it more than just a calculator is the intelligence layer — the system does not simply report numbers, it interprets them and turns them into advice a student can act on immediately.

The project demonstrates how Agentic AI architectures can be applied to real-world educational problems in a lightweight and accessible way. The chat interface lowers the barrier to entry, the agent framework keeps the system flexible, and the Groq LLM makes the output genuinely readable. Together these components produce something that feels like a knowledgeable academic assistant rather than an automated script.

Most importantly, this project addresses a real gap — students who lack access to personalized academic guidance can now get a structured, data-driven analysis of their own performance in seconds, from any browser, without needing to speak to anyone.

Future Scope

Several improvements would make this system significantly more powerful in future versions:

- Integration with institutional databases or Learning Management Systems for automatic data retrieval
- Historical performance tracking so students can see their progress across multiple semesters on a dashboard
- Subject-level AI tutoring where the system recommends specific study resources for each weak area
- Automated alerts that notify students when their attendance or marks drop below a defined threshold
- A mobile application version for greater accessibility on smartphones
- Multi-student dashboards for teachers and academic coordinators to monitor class-wide trends

GITHUB: <https://github.com/Sudheer759/student-performance-analyzer>